

UNIT -3

C# operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise etc.

There are following types of operators to perform different types of operations in C# language.

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Unary Operators
- Ternary Operators

- Misc Operators
- Misc Operators

	Operator	Type
Binary Operator	+, -, *, /, %	Arithmetic Operators
	<, <=, >, >=, ==, !=	Relational Operators
	&&, , !	Logical Operators
	&, , <<, >>, ~, ^	Bitwise Operators
	=, +=, -=, *=, /=, %=	Assignment Operators
Unary Operator	++, --	Unary Operator
Ternary Operator	?:	Ternary or Conditional Operator

Precedence of Operators in C#

The precedence of operator specifies that which operator will be evaluated first and next. The associativity specifies the operators direction to be evaluated, it may be left to right or right to left.

Let's understand the precedence by the example given below:



1. `int data= 10+ 5*5`

The "data" variable will contain 35 because * (multiplicative operator) is evaluated before + (additive operator).

The precedence and associativity of C# operators is given below:

Category (By Precedence)	Operator(s)	Associativity
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to Left
Additive	+ -	Left to Right
Multiplicative	% / *	Left to Right
Relational	< > <= >=	Left to Right
Shift	<< >>	Left to Right
Equality	== !=	Right to Left
Logical AND	&	Left to Right
Logical OR		Left to Right
Logical XOR	^	Left to Right
Conditional OR		Left to Right

Conditional AND	&&	Left to Right
Null Coalescing	??	Left to Right
Ternary	?:	Right to Left
Assignment	= *= /= %= += -= <<= >>= &= ^= = =>	Right to Left

1. Basic Assignment Operator

Basic assignment operator (=) is used to assign values to variables. For example,

```
double x;

x = 50.05;
```

Here, 50.05 is assigned to x.

1: Basic Assignment Operator

```
using System;

namespace Operator
{
    class AssignmentOperator
    {
        public static void Main(string[] args)
        {
            int firstNumber, secondNumber;
            // Assigning a constant to variable
            firstNumber = 10;
            Console.WriteLine("First Number = {0}", firstNumber);
        }
    }
}
```

```
        // Assigning a variable to another variable
        secondNumber = firstNumber;
        Console.WriteLine("Second Number = {0}", secondNumber);
    }
}
```

When we run the program, the output will be:

```
First Number = 10
Second Number = 10
```

2. Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations such as addition, subtraction, multiplication, division, etc.

For example,

```
int x = 5;

int y = 10;

int z = x + y; // z = 15
```

C# Arithmetic Operators

Operator	Operator Name	Example
+	Addition Operator	6 + 3 evaluates to 9
-	Subtraction Operator	10 - 6 evaluates to 4
*	Multiplication Operator	4 * 2 evaluates to 8

C# Arithmetic Operators

Operator	Operator Name	Example
/	Division Operator	10 / 5 evaluates to 2
%	Modulo Operator (Remainder)	16 % 3 evaluates to 1

2: Arithmetic Operators

```
using System;

namespace Operator
{
    class ArithmeticOperator
    {
        public static void Main(string[] args)
        {
            double firstNumber = 14.40, secondNumber = 4.60, result;
            int num1 = 26, num2 = 4, rem;

            // Addition operator
            result = firstNumber + secondNumber;
            Console.WriteLine("{0} + {1} = {2}", firstNumber, secondNumber, result);

            // Subtraction operator
            result = firstNumber - secondNumber;
            Console.WriteLine("{0} - {1} = {2}", firstNumber, secondNumber, result);

            // Multiplication operator
            result = firstNumber * secondNumber;
            Console.WriteLine("{0} * {1} = {2}", firstNumber, secondNumber, result);

            // Division operator
            result = firstNumber / secondNumber;
            Console.WriteLine("{0} / {1} = {2}", firstNumber, secondNumber, result);

            // Modulo operator
            rem = num1 % num2;
            Console.WriteLine("{0} % {1} = {2}", num1, num2, rem);
        }
    }
}
```

When we run the program, the output will be:

```
14.4 + 4.6 = 19
14.4 - 4.6 = 9.8
14.4 * 4.6 = 66.24
14.4 / 4.6 = 3.1304347826087
26 % 4 = 2
```

Arithmetic operations are carried out in the above example. Variables can be replaced by constants in the statements. For example,

```
result = 4.5 + 2.7 ; // result will hold 7.2

result = firstNumber - 3.2; // result will hold 11.2
```

3. Relational Operators

Relational operators are used to check the relationship between two operands. If the relationship is true the result will be `true`, otherwise it will result in `false`. Relational operators are used in decision making and loops.

C# Relational Operators

Operator	Operator Name	Example
<code>==</code>	Equal to	<code>6 == 4</code> evaluates to false
<code>></code>	Greater than	<code>3 > -1</code> evaluates to true
<code><</code>	Less than	<code>5 < 3</code> evaluates to false
<code>>=</code>	Greater than or equal to	<code>4 >= 4</code> evaluates to true

C# Relational Operators

Operator	Operator Name	Example
<=	Less than or equal to	5 <= 3 evaluates to false
!=	Not equal to	10 != 2 evaluates to true

3: Relational Operators

```
using System;

namespace Operator
{
    class RelationalOperator
    {
        public static void Main(string[] args)
        {
            bool result;
            int firstNumber = 10, secondNumber = 20;

            result = (firstNumber == secondNumber);
            Console.WriteLine("{0} == {1} returns {2}", firstNumber, secondNumber,
result);

            result = (firstNumber > secondNumber);
            Console.WriteLine("{0} > {1} returns {2}", firstNumber, secondNumber,
result);

            result = (firstNumber < secondNumber);
            Console.WriteLine("{0} < {1} returns {2}", firstNumber, secondNumber,
result);

            result = (firstNumber >= secondNumber);
            Console.WriteLine("{0} >= {1} returns {2}", firstNumber, secondNumber,
result);

            result = (firstNumber <= secondNumber);
            Console.WriteLine("{0} <= {1} returns {2}", firstNumber, secondNumber,
result);

            result = (firstNumber != secondNumber);
            Console.WriteLine("{0} != {1} returns {2}", firstNumber, secondNumber,
result);
        }
    }
}
```

```
}  
}
```

When we run the program, the output will be:

```
10 == 20 returns False  
10 > 20 returns False  
10 < 20 returns True  
10 >= 20 returns False  
10 <= 20 returns True  
10 != 20 returns True
```

4. Logical Operators

Logical operators are used to perform logical operation such as and, or. Logical operators operate on boolean expressions (true and false) and return boolean values. Logical operators are used in decision making and loops.

Here is how the result is evaluated for logical AND and OR operators.

C# Logical operators

Operand 1	Operand 2	OR ()	AND (&&)
true	true	true	true
true	false	true	false
false	true	true	false
false	false	false	false

In simple words, the table can be summarized as:

- If one of the operand is true, the OR operator will evaluate it to true.
- If one of the operand is false, the AND operator will evaluate it to false.

4: Logical Operators

```
using System;

namespace Operator
{
    class LogicalOperator
    {
        public static void Main(string[] args)
        {
            bool result;
            int firstNumber = 10, secondNumber = 20;

            // OR operator
            result = (firstNumber == secondNumber) || (firstNumber > 5);
            Console.WriteLine(result);

            // AND operator
            result = (firstNumber == secondNumber) && (firstNumber > 5);
            Console.WriteLine(result);
        }
    }
}
```

When we run the program, the output will be:

```
True
False
```

5. Unary Operators

Unlike other operators, the unary operators operates on a single operand.

C# unary operators

Operator	Operator Name	Description
+	Unary Plus	Leaves the sign of operand as it is
-	Unary Minus	Inverts the sign of operand
++	Increment	Increment value by 1
--	Decrement	Decrement value by 1
!	Logical Negation (Not)	Inverts the value of a boolean

Example 5: Unary Operators

```
using System;

namespace Operator
{
    class UnaryOperator
    {
        public static void Main(string[] args)
        {
            int number = 10, result;
            bool flag = true;

            result = +number;
            Console.WriteLine("+number = " + result);

            result = -number;
            Console.WriteLine("-number = " + result);

            result = ++number;
            Console.WriteLine("++number = " + result);

            result = --number;
            Console.WriteLine("--number = " + result);

            Console.WriteLine("!flag = " + (!flag));
        }
    }
}
```

```
}
```

When we run the program, the output will be:

```
+number = 10  
-number = -10  
++number = 11  
--number = 10  
!flag = False
```

The increment (++) and decrement (--) operators can be used as prefix and postfix. If used as prefix, the change in value of variable is seen on the same line and if used as postfix, the change in value of variable is seen on the next line. This will be clear by the example below.

Bitwise and Bit Shift Operators

Bitwise and bit shift operators are used to perform bit manipulation operations.

C# Bitwise and Bit Shift operators

Operator	Operator Name
~	Bitwise Complement
&	Bitwise AND
	Bitwise OR
^	Bitwise Exclusive OR
<<	Bitwise Left Shift
>>	Bitwise Right Shift

Example 8: Bitwise and Bit Shift Operator

```
using System;  
  
namespace Operator
```

```

{
    class BitOperator
    {
        public static void Main(string[] args)
        {
            int firstNumber = 10;
            int secondNumber = 20;
            int result;

            result = ~firstNumber;
            Console.WriteLine("~{0} = {1}", firstNumber, result);

            result = firstNumber & secondNumber;
            Console.WriteLine("{0} & {1} = {2}", firstNumber, secondNumber, result);

            result = firstNumber | secondNumber;
            Console.WriteLine("{0} | {1} = {2}", firstNumber, secondNumber, result);

            result = firstNumber ^ secondNumber;
            Console.WriteLine("{0} ^ {1} = {2}", firstNumber, secondNumber, result);

            result = firstNumber << 2;
            Console.WriteLine("{0} << 2 = {1}", firstNumber, result);

            result = firstNumber >> 2;
            Console.WriteLine("{0} >> 2 = {1}", firstNumber, result);
        }
    }
}

```

When we run the program, the output will be:

```

~10 = -11
10 & 20 = 0
10 | 20 = 30
10 ^ 20 = 30
10 << 2 = 40
10 >> 2 = 2

```

Some Special Operators in C#

The Checked and Unchecked Operator

C# provides special operators, checked and unchecked. It enforces CLR to check overflow.

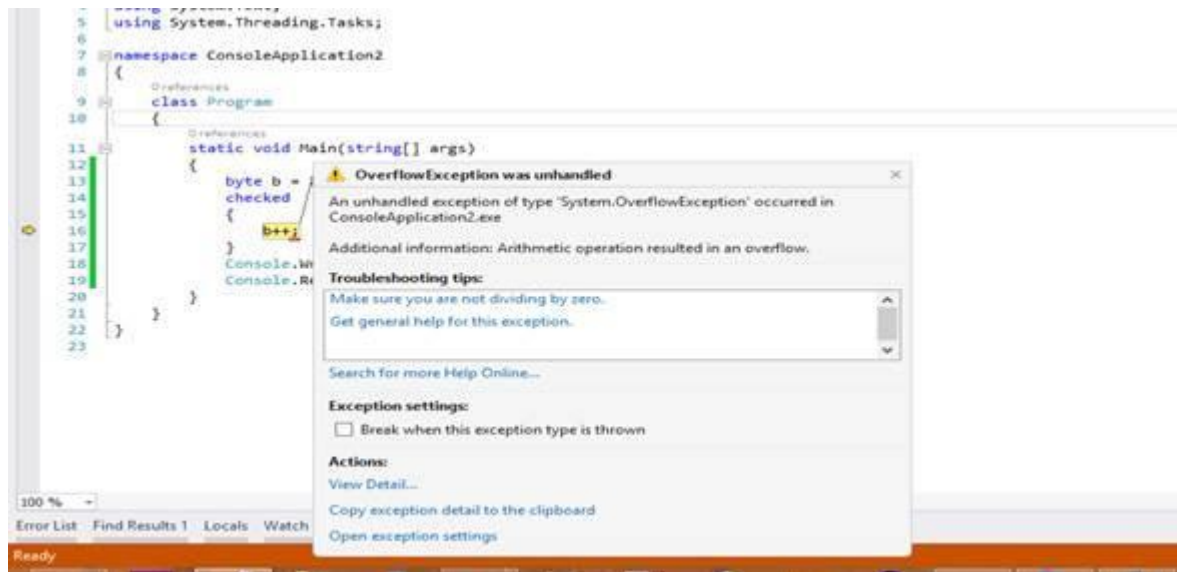
Consider the following code

```
1. class Program
2. {
3.     static void Main(string[] args)
4.     {
5.         byte b = 255;
6.
7.         b++;
8.
9.         Console.WriteLine(b);
10.        Console.ReadKey();
11.    }
12. }
```



Byte data type range is 0 to 255 and in this code when we try to increase value then it loses value and it produces 0 as output. We have to find out any method through which we can get the exact value.

To do this we used checked operator. Consider the following code.



```
1. class Program
2. {
3.     static void Main(string[] args)
4.     {
5.         byte b = 255;
6.         checked
7.         {
8.             b++;
9.         }
10.        Console.WriteLine(b);
11.        Console.ReadKey();
12.    }
13. }
```

When we try to run this code, we will get an error.

Unchecked is the default behavior it will produce 0 as out put,

```
1. unchecked
2. {
3. b++;
4. }
```

The is Operator

With the help of “is” operator we can check if an object is compatible with a specific type. The is operator returns a Boolean value of true or false.

Consider following code

```
1. namespace ConsoleApplication2
2. {
3.     class Program
4.     {
5.         static void Main(string[] args)
6.         {
7.             int i = 10;
8.             if(i is string)
9.             {
10.                Console.WriteLine("string type");
11.
12.            }
13.            else if (i is float )
14.            {
15.                Console.WriteLine("float type");
16.            }
17.            else if(i is int)
18.            {
19.                Console.WriteLine("Int type");
20.            }
21.            Console.ReadKey();
22.        }
23.    }
24. }
```

It will produce “Int type” as output

The as Operator

It is used to perform explicit type conversion of reference type.

If type converted is compatible with specified type then conversion is performed successfully otherwise it assigns null value. For example consider the following code.

```
1. namespace ConsoleApplication2
2. {
3.     class Program
4.     {
5.         static void Main(string[] args)
6.         {
7.             object o1 = "C# corner ";
8.             object o2 = 5;
9.             string s1=o1 as string;
10.            string s2 = o2 as string;
11.            Console.WriteLine(s1);
12.            Console.WriteLine(s2);
13.            Console.ReadKey();
14.        }
15.    }
16. }
```

What is C# Expression

Many times while reading a C# book or reading any article you may encounter a word named expression. And you may start wondering what exactly is expression. In this article I will help you to understand what an expression is and different types of expressions.

C# expression

Lets see the official definition from the Microsoft website.

“An *expression* is a sequence of one or more operands and zero or more operators that can be evaluated to a single value, object, method, or namespace. “

Let simplify it and just say that a C# expression is a value. The simplest type of expression are constants and variables. Expressions can be combined using operators and transformed as well.

Below is an example of constant expression

12

We can use the `*` operator to combine two operands (the literal expressions 12 and 30), as follows:

12 * 30

Complex expressions can be built because an operand may itself be an expression, such as the operand (12 * 30) in the following example:

1 + (12 * 30)

Operators in C# can be classed as *unary*, *binary*, or *ternary*—depending on the number of operands they work on (one, two, or three). The binary operators always use *infix* notation, where the operator is placed *between* the two operands.

Primary Expression

There are the expression which are defined by the operators belonging to the language and framework. For example, have a look at the below code

```
Math.Pow(2, 2);
```

In the above code there are two operators. First one is the `.` (dot) operator which looks up for the `Pow` method defined in the `Math` class. This is expression performs a method look up. Other expression performs method call.

Void Expressions

A void expression is an expression that has no value. For example:

```
Console.WriteLine (1)
```

A void expression, since it has no value, cannot be used as an operand to build more complex expressions:

```
1 + Console.WriteLine (1)    // Compile-time error
```

Assignment Expressions

An assignment expression uses the = operator to assign the result of another expression to a variable. For example:

```
x = x * 5
```

An assignment expression is not a void expression—it has a value of whatever was assigned, and so can be incorporated into another expression. In the following example, the expression assigns 2 to x and 10 to y:

```
y = 5 * (x = 2)
```

This style of expression can be used to initialize multiple values:

```
a = b = c = d = 0
```

The compound assignment operators are syntactic shortcuts that combine assignment with another operator. For example:

```
x *= 2    // equivalent to x = x * 2  
x <<= 1   // equivalent to x = x << 1
```

Expression Evaluation

Evaluate an expression represented by a String. The expression can contain parentheses, you can assume parentheses are well-matched. For simplicity, you can

assume only binary operations allowed are +, -, *, and /. Arithmetic Expressions can be written in one of three forms:

- **Infix Notation:** Operators are written between the operands they operate on, e.g. $3 + 4$.
- **Prefix Notation:** Operators are written before the operands, e.g. $+ 3 4$
- **Postfix Notation:** Operators are written after operands.

Infix Expressions are harder for Computers to evaluate because of the additional work needed to decide precedence. Infix notation is how expressions are written and recognized by humans and, generally, input to programs. Given that they are harder to evaluate, they are generally converted to one of the two remaining forms. A very well known algorithm for converting an infix notation to a postfix notation is Shunting Yard Algorithm by Edgar Dijkstra.

1. While there are still tokens to be read in,

1.1 Get the next token.

1.2 If the token is:

1.2.1 A number: push it onto the value stack.

1.2.2 A variable: get its value, and push onto the value stack.

1.2.3 A left parenthesis: push it onto the operator stack.

1.2.4 A right parenthesis:

1 While the thing on top of the operator stack is not a left parenthesis,

1 Pop the operator from the operator stack.

2 Pop the value stack twice, getting two operands.

3 Apply the operator to the operands, in the correct order.

4 Push the result onto the value stack.

2 Pop the left parenthesis from the operator stack, and discard it.

1.2.5 An operator (call it thisOp):

1 While the operator stack is not empty, and the top thing on the operator stack has the same or greater precedence as thisOp,

1 Pop the operator from the operator stack.

2 Pop the value stack twice, getting two operands.

3 Apply the operator to the operands, in the correct order.

4 Push the result onto the value stack.

- 2 Push thisOp onto the operator stack.
2. While the operator stack is not empty,
 - 1 Pop the operator from the operator stack.
 - 2 Pop the value stack twice, getting two operands.
 - 3 Apply the operator to the operands, in the correct order.
 - 4 Push the result onto the value stack.
3. At this point the operator stack should be empty, and the value stack should have only one value in it, which is the final result.

Implementation: It should be clear that this algorithm runs in linear time – each number or operator is pushed onto and popped from Stack only once.

C# Operators Precedence

In C#, operator precedence determines the order in which operators are evaluated in an expression, and associativity defines the order in which operators of the same precedence are evaluated. Here is a summary of the operator precedence and associativity in C#:

Operator Precedence (from highest to lowest):

1. **Postfix operators:** `x++, x--`
2. **Unary operators:** `+, -, !x, ~x, ++x, --x, (T)x, await, sizeof, typeof, checked, unchecked`
3. **Multiplicative operators:** `*, /, %`
4. **Additive operators:** `+, -`
5. **Shift operators:** `<<, >>`
6. **Relational and type testing operators:** `<, >, <=, >=, is, as`
7. **Equality operators:** `==, !=`
8. **Logical AND operator:** `&&`
9. **Logical OR operator:** `||`
10. **Null-coalescing operator:** `??`
11. **Conditional operator:** `? :`
12. **Assignment operators:** `=, +=, -=, *=, /=, %=` etc.
13. **Lambda operator:** `=>`

Associativity:

- Most binary operators are left-associative, meaning they are evaluated from left to right. For example, `x + y + z` is equivalent to `(x + y) + z`.

- The assignment operators (`=`, `+=`, `-=` etc.) are right-associative, meaning they are evaluated from right to left. For example, `x = y = z` is equivalent to `x = (y = z)`.

It's important to be aware of operator precedence and associativity to avoid unexpected behavior in your code. You can use parentheses to explicitly specify the order of evaluation when needed.

Operator precedence (**order of operations**) is a collection of rules that reflect conventions about which procedures to perform first in order to evaluate a given expression.

For example, multiplication has higher precedence than addition. Thus, the expression $1 + 2 \times 3$ is interpreted to have the value $1 + (2 \times 3) = 7$, and not $(1 + 2) \times 3 = 9$. When exponent is used in the expression, it has precedence over both addition and multiplication. Thus $3 + 5^2 = 28$ and $3 \times 5^2 = 75$.

Operator Associativity

Operator associativity is the direction in which an expression is evaluated. For example:

```
int x = 10;
int y = 20;

//associativity of = operator is
//Left to Right, hence x will become 20
x = y;
```

As the associativity of `=` is left to right. Hence `x` is assigned the value of `y`.

Example:

The example below illustrates the operator precedence in C#.

```
using System;

class MyProgram {
    static void Main(string[] args) {
        int retval1, retval2, retval3;

        //evaluates 5 * 2 first
        retval1 = 15 - 5 * 2;

        //above expression is equivalent to
        retval2 = 15 - (5 * 2);

        //forcing compiler to evaluate 15 - 5 first
        retval3 = (15 - 5) * 2;
```

```
Console.WriteLine("15 - 5 * 2 = " + retval1);
Console.WriteLine("15 - (5 * 2) = " + retval2);
Console.WriteLine("(15 - 5) * 2 = " + retval3);
}
}
```

The output of the above code will be:

```
15 - 5 * 2 = 5
15 - (5 * 2) = 5
(15 - 5) * 2 = 20
```

C# Operators Precedence Table

The following table lists the precedence and associativity of C# operators. Operators are listed top to bottom, in **descending precedence**. Operators with higher precedence are evaluated before operators with relatively lower precedence. When operators have the same precedence, associativity of the operators determines the order in which the operations are performed.

#1 Operator Precedence in C# Programming

Code - C# Programming

```
using System;

namespace Operator
{
    class OperatorPrecedence
    {
        public static void Main(string[] args)
        {
            int result1;
            int a = 5, b = 6, c = 4;
            result1 = --a * b - ++c;
            Console.WriteLine(result1);

            bool result2;
            result2 = b >= c + a;
            Console.WriteLine(result2);
        }
    }
}
```

```
    }  
    }  
}
```

Output

```
19  
False
```

ExP---Associativity of Operators in C# Programming

```
using System;  
namespace Operator  
{  
    class OperatorPrecedence  
    {  
        public static void Main(string[] args)  
        {  
            int a = 5, b = 6, c = 3;  
            int result = a * b / c;  
            Console.WriteLine(result);  
  
            a = b = c;  
            Console.WriteLine("a = {0}, b = {1}, c = {2}", a, b, c);  
        }  
    }  
}
```

Output

```
10  
a = 3, b = 3, c = 3
```

Example

```
using System;
```

```

namespace OperatorsAppl {
    class Program {
        static void Main(string[] args) {
            int a = 20;
            int b = 10;
            int c = 15;
            int d = 5;
            int e;
            e = (a + b) * c / d;    // ( 30 * 15 ) / 5
            Console.WriteLine("Value of (a + b) * c / d is : {0}", e);

            e = ((a + b) * c) / d; // (30 * 15) / 5
            Console.WriteLine("Value of ((a + b) * c) / d is : {0}", e);

            e = (a + b) * (c / d); // (30) * (15/5) {0}", e);

            e = a + (b * c) / d;   // 20 + (150/5) {0}", e);
            Console.ReadLine();
        }
    }
}

```

Following Result –

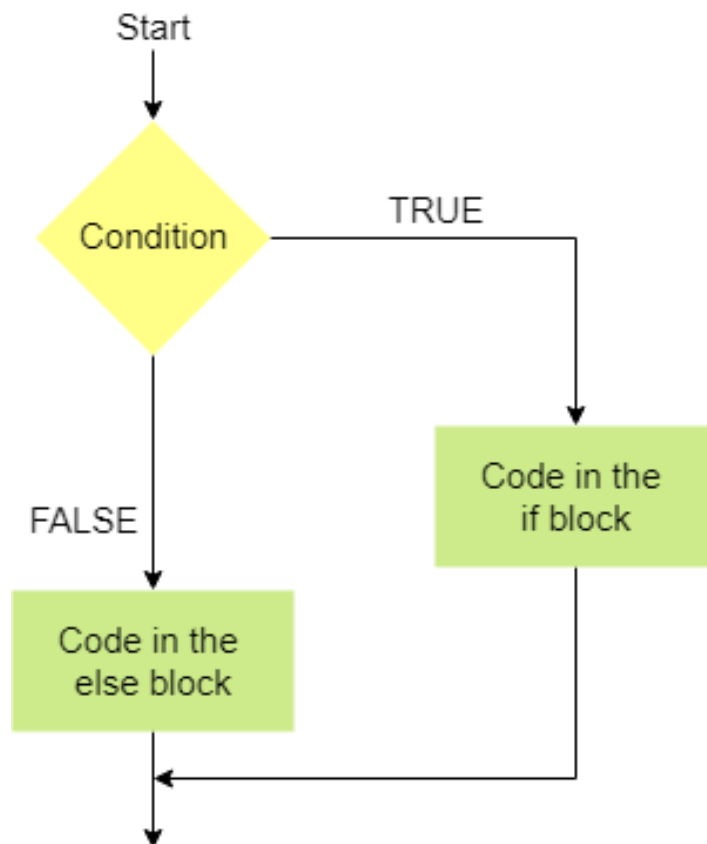
Value of (a + b) * c / d is : 90
 Value of ((a + b) * c) / d is : 90

Control Statements in C# - if, else, Switch, LOOP

1. **if...else** statement
2. **if...else if...else** statements
3. **switch** statement
4. **goto** statement
5. **Loop**

If, If-else, Switch Case Control Statements in C#

1. if...else statement



The if statement is used to check for a condition and based on it decide whether or not to execute a code block. The if code block is executed only if the condition is **true** otherwise further instruction or else block gets executed. You can write nested if blocks if required. The following diagram shows the basic flow of a default if-else statement. It is not mandatory to have an else block after an if block. The else statement is used to specify the code which has to be executed if the condition doesn't return true.

Let's see a practical implementation of if statement.

```
using System;

namespace Studytonight
{
    public class Program
```

```
{  
  
    public static void Main(string[] args)  
  
    {  
  
        int i = 100;  
  
        if(i > 0)  
  
        {  
  
            Console.WriteLine("Given number is positive!");  
  
        }  
  
        else  
  
        {  
  
            Console.WriteLine("Given number is negative!");  
  
        }  
  
        Console.ReadKey();  
  
    }  
  
}
```

This program checks if the value of *i* is greater than 0 using an if statement.

If the condition *i* > 0 evaluates to true, the code inside the if block will be executed. In this case, it prints the message "Given number is positive!" and If the condition *i* > 0 evaluates to false, the code inside the else block will be executed. It prints the message "Given number is negative!" to the console.

Output:

```
Given number is positive!
```

Try running the above program by providing a negative value to *i* variable, the *else* code will get executed.

2. if...else if...else statement

If we have to specify two conditions that are related to each other, we can use the else if statement to specify the second condition and its related code block. We can use multiple else if blocks to add multiple conditions but it requires at least one if block at the beginning, we can't directly write else and else if statements without having any if block.

```
using System;

namespace Studytonight
{
    public class Program
    {
        public static void Main(string[] args)
        {
            int i = 0; // Zero is neutral in nature

            if(i > 0)
            {
                Console.WriteLine("Given number is positive!");
            }

            else if(i < 0)
            {
                Console.WriteLine("Given number is negative!");
            }

            else
            {
                Console.WriteLine("Given number is neither positive nor negative!");
            }
        }
    }
}
```

```
    }  
  
    Console.ReadKey();  
  
    }  
  
    }  
  
}
```

The first condition `if (i > 0)` checks if the value of `i` is greater than 0. If it evaluates to true, the code inside the `if` block will be executed.

If the first condition `if (i > 0)` evaluates to false, the program proceeds to the next condition which checks if the value of `i` is less than 0. If it evaluates to true, the code inside the `else if` block will be executed and will print the message "Given number is negative!" to the console.

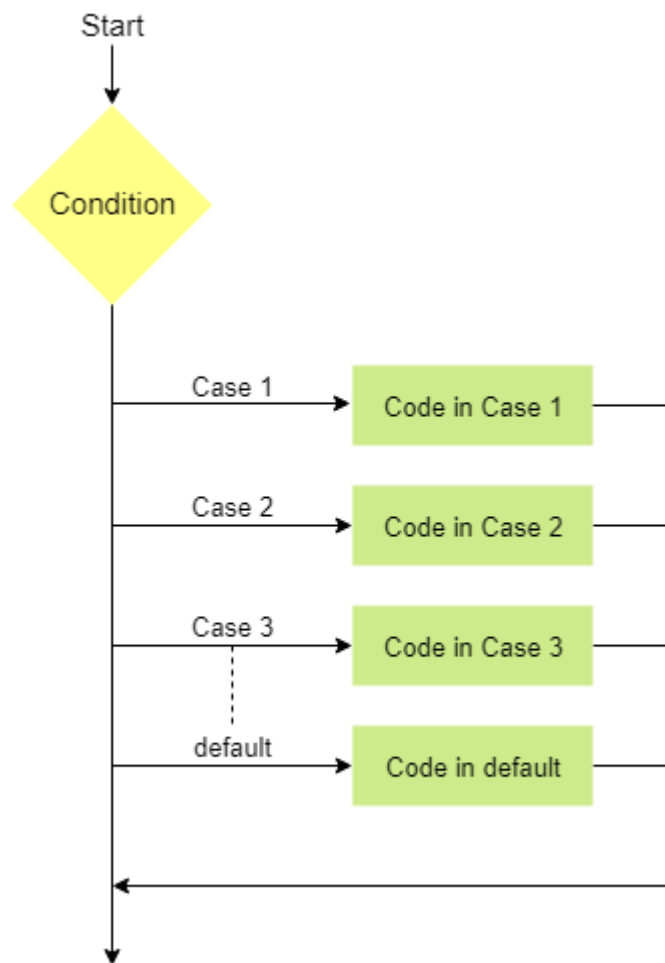
If both the conditions `if (i > 0)` and `else if (i < 0)` evaluate to false, **it means that the value of `i` is 0**. In this case, the code inside the `else` block will be executed.

Output, it means that the value of `i` is 0. Output:

Given number is neither positive nor negative!

3. switch statement

The switch statement executes only one statement from multiple given statements associated with conditions. If any condition returns true, then the code statements below it get executed. The following diagram shows the basic flow of a switch statement.



Let's see the practical implementation of the switch statement,

```
using System;

namespace Studytonight
{
    public class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("1.Addition\n2.Subtraction\n3.Division\nEnter your choice:");

            int c = Convert.ToInt32(Console.ReadLine());
```

```

switch(c)
{
    case 1: Console.WriteLine("\nAddition selected!"); break;

    case 2: Console.WriteLine("\nSubtraction selected!"); break;

    case 3: Console.WriteLine("\nDivision selected!"); break;

    default: Console.WriteLine("\nYour choice not found!"); break;

}

Console.ReadKey();

}

}

```

The code uses a switch statement to perform different actions based on the value of c. If c matches the value 1, the code inside case 1: block is executed and will print the message "\nAddition selected!" to the console.

If c matches the value 2, the code inside case 2: block is executed and it prints the message "\nSubtraction selected!" to the console. If the value is 3, the code inside case 3: block is executed and If none of the case labels match the value of c, the code inside the default: block is executed.

Output:

```

1.Additon
2.Subtraction
3.Division
Enter your choice: 4

Your choice not found!

```

As you can see in the above code example, if none of the cases is matched, then the **default case** will be executed. If your choice is **1** then only **case 1** will be executed and other cases will not execute not even the default case. The **break** statement is used to come out of any switch case, if we remove the **break** statements from all the case blocks, then after the execution of the case for which the condition is true, the default condition will also get executed.

4. goto statement

The goto statement is used to transfer the control from a loop or switch case **to a specified label**. It is also known as a **jump statement**. It is generally recommended to avoid using it as it makes the program more complex. In the following example, if the user is logged in then we have directly jumped our code to the **success** label.

```
using System;

namespace Studytonight
{
    public class Program
    {
        public static void Main(string[] args)
        {
            string user = "login";

            if(user=="login")
            {
                goto success;
            }
        }
    }
}
```

```
success:

Console.WriteLine("Welcome to Studytonight!");

Console.ReadKey();

}

}

}
```

Output:

```
Welcome to Studytonight!
```

for loop

It executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.

The following is the syntax –

```
for ( init; condition; increment ) {
    statement(s);
}
```

while loop

It repeats a statement or a group of statements while a given condition is true. It tests the condition before executing the loop body.

The following is the syntax –

```
while(condition) {
    statement(s);
}
```



```
}
```

do...while loop

It is similar to a while statement, except that it tests the condition at the end of the loop body.

The following is the syntax –

```
do {  
    statement(s);  
} while( condition );
```

C# Object and Class

Since C# is an object-oriented language, program is designed using objects and classes in C#.

C# Object

In C#, Object is a real world entity, for example, chair, car, pen, mobile, laptop etc.

In other words, object is an entity that has state and behavior. Here, **state means data and behavior means functionality.**

Object is a runtime entity, it is created at runtime.

Object is an instance of a class. All the members of the class can be accessed through object.

Let's see an example to create object using new keyword.

1. Student s1 = **new** Student();//creating an object of Student

In this example, Student is the type and s1 is the reference variable that refers to the instance of Student class. The new keyword allocates memory at runtime.

C# Class

In C#, class is a group of similar objects. It is a template from which objects are created. It can have fields, methods, constructors etc.

Let's see an example of C# class that has two fields only.

```
1. public class Student
2. {
3.     int id;//field or data member
4.     String name;//field or data member
5. }
```

C# Object and Class Example

Let's see an example of class that has two fields: id and name. It creates instance of the class, initializes the object and prints the object value.

```
1. using System;
2. public class Student
3. {
4.     int id;//data member (also instance variable)
5.     String name;//data member(also instance variable)
6.
7.     public static void Main(string[] args)
8.     {
9.         Student s1 = new Student();//creating an object of Student
10.        s1.id = 101;
11.        s1.name = "Sonoo Jaiswal";
12.        Console.WriteLine(s1.id);
13.        Console.WriteLine(s1.name);
14.
15.    }
16. }
```

Output:

```
101
Sonoo Jaiswal
```

C# Class Example 2: Having Main() in another class

Let's see another example of class where we are having Main() method in another class. In such case, class must be public.

```
1. using System;
2. public class Student
3. {
4.     public int id;
```

```

5.     public String name;
6. }
7. class TestStudent{
8.     public static void Main(string[] args)
9.     {
10.         Student s1 = new Student();
11.         s1.id = 101;
12.         s1.name = "Sonoo Jaiswal";
13.         Console.WriteLine(s1.id);
14.         Console.WriteLine(s1.name);
15.
16.     }
17. }

```

Output:

```

101
Sonoo Jaiswal

```

C# Class Example 3: Initialize and Display data through method

Let's see another example of C# class where we are initializing and displaying object through method.

```

1. using System;
2. public class Student
3. {
4.     public int id;
5.     public String name;
6.     public void insert(int i, String n)
7.     {
8.         id = i;
9.         name = n;
10.    }
11.    public void display()
12.    {
13.        Console.WriteLine(id + " " + name);
14.    }
15. }
16. class TestStudent{
17.     public static void Main(string[] args)
18.     {
19.         Student s1 = new Student();
20.         Student s2 = new Student();
21.         s1.insert(101, "Ajeet");
22.         s2.insert(102, "Tom");

```

```

23.     s1.display();
24.     s2.display();
25.
26. }
27. }

```

Output:

```

101 Ajeet
102 Tom

```

C# Class Example 4: Store and Display Employee Information

```

1. using System;
2. public class Employee
3. {
4.     public int id;
5.     public String name;
6.     public float salary;
7.     public void insert(int i, String n,float s)
8.     {
9.         id = i;
10.        name = n;
11.        salary = s;
12.    }
13.    public void display()
14.    {
15.        Console.WriteLine(id + " " + name+" "+salary);
16.    }
17. }
18. class TestEmployee{
19.     public static void Main(string[] args)
20.     {
21.         Employee e1 = new Employee();
22.         Employee e2 = new Employee();
23.         e1.insert(101, "Sonoo",890000f);
24.         e2.insert(102, "Mahesh", 490000f);
25.         e1.display();
26.         e2.display();
27.
28.     }
29. }

```

Output:

```

101 Sonoo 890000
102 Mahesh 490000

```

C# - Methods

A **method** is a block of code which only runs when it is called.

You can pass data, known as parameters, into a method.

Methods are used to perform certain actions, and they are also known as **functions**.

Why use methods? To reuse code: define the code once, and use it many times.

A method is a group of statements that together perform a task. Every C# program has at least one class with a method named Main.

To use a method, you need to –

- Define the method
- Call the method

Defining Methods in C#

When you define a method, you basically declare the elements of its structure.

The syntax for defining a method in C# is as follows –

```
<Access Specifier> <Return Type> <Method Name>(Parameter List) {  
    Method Body  
}
```

Following are the various elements of a method –

- **Access Specifier** – This determines the visibility of a variable or a method from another class.
- **Return type** – A method may return a value. The return type is the data type of the value the method returns. If the method is not returning any values, then the return type is **void**.
- **Method name** – Method name is a unique identifier and it is case sensitive. It cannot be same as any other identifier declared in the class.

- **Parameter list** – Enclosed between parentheses, the parameters are used to pass and receive data from a method. The parameter list refers to the type, order, and number of the parameters of a method. Parameters are optional; that is, a method may contain no parameters.
- **Method body** – This contains the set of instructions needed to complete the required activity.

Example

Following code snippet shows a function *FindMax* that takes two integer values and returns the larger of the two. It has public access specifier, so it can be accessed from outside the class using an instance of the class.

```
class NumberManipulator {  
  
    public int FindMax(int num1, int num2) {  
        /* local variable declaration */  
        int result;  
  
        if (num1 > num2)  
            result = num1;  
        else  
            result = num2;  
  
        return result;  
    }  
    ...  
}
```

Calling Methods in C#

You can call a method using the name of the method. The following example illustrates this –

```
using System;  
  
namespace CalculatorApplication {  
    class NumberManipulator {  
        public int FindMax(int num1, int num2) {  
            /* local variable declaration */  
            int result;  
  
            if (num1 > num2)
```

```

        result = num1;
    else
        result = num2;
    return result;
}

static void Main(string[] args) {
    /* local variable definition */
    int a = 100;
    int b = 200;
    int ret;
    NumberManipulator n = new NumberManipulator();

    //calling the FindMax method
    ret = n.FindMax(a, b);
    Console.WriteLine("Max value is : {0}", ret );
    Console.ReadLine();
}
}
}

```

When the above code is compiled and executed, it produces the following result –

Max value is : 200

You can also call public method from other classes by using the instance of the class. For example, the method *FindMax* belongs to the *NumberManipulator* class, you can call it from another

```

using System;

namespace CalculatorApplication {
    class NumberManipulator {
        public int FindMax(int num1, int num2) {
            /* local variable declaration */
            int result;

            if(num1 > num2)
                result = num1;
            else
                result = num2;
        }
    }
}

```

```

        return result;
    }
}
class Test {
    static void Main(string[] args) {
        /* local variable definition */
        int a = 100;
        int b = 200;
        int ret;
        NumberManipulator n = new NumberManipulator();

        //calling the FindMax method
        ret = n.FindMax(a, b);
        Console.WriteLine("Max value is : {0}", ret );
        Console.ReadLine();
    }
}

```

When the above code is compiled and executed, it produces the following result –

Max value is : 200

C# | Constructors

A constructor is a special method of the class which gets automatically invoked whenever an instance of the class is created. Like methods, a constructor also contains the collection of instructions that are executed at the time of Object creation. It is used to assign initial values to the **data members** of the same class.

Example :

```

class Geek
{
    .....
    // Constructor
    public Geek() {}
    .....
}

```



```
// an object is created of Geek class,  
// So above constructor is called  
Geek obj = new Geek();
```

Important points to Remember About Constructors

- Constructor of a class must have the same name as the class name in which it resides.
- A constructor can not be abstract, final, and Synchronized.
- Within a class, you can create only one static constructor.
- A constructor doesn't have any return type, not even void.
- A static constructor cannot be a parameterized constructor.
- A class can have any number of constructors.
- Access modifiers can be used in constructor declaration to control its access i.e which other class can call the constructor.

Types of Constructor

1. **Default Constructor**
2. **Parameterized Constructor**
3. **Copy Constructor**
4. **Private Constructor**
5. **Static Constructor**

Default Constructor

A constructor with no parameters is called a default constructor. A default constructor has every instance of the class to be initialized to the same values. The default constructor initializes all numeric fields to zero and all string and object fields to null inside a class.

Example :

```
// C# Program to illustrate calling
// a Default constructor

using System;

namespace DefaultConstructorExample {

class Geek {

    int num;

    string name;

    // this would be invoked while the
    // object of that class created.

    Geek()

    {

        Console.WriteLine("Constructor Called");

    }

    // Main Method

    public static void Main()

    {

        // this would invoke default
        // constructor.

    }

}
```

```
Geek geek1 = new Geek();

// Default constructor provides

// the default values to the

// int and object as 0, null

// Note:

// It Give Warning because

// Fields are not assign

Console.WriteLine(geek1.name);

Console.WriteLine(geek1.num);

}

}

}
```

Output :

Constructor Called

0

Parameterized Constructor

A constructor having at least one parameter is called as parameterized constructor. It can initialize each instance of the class to different values.

Example :

C#

```
// C# Program to illustrate calling of
```

```
// parameterized constructor.
```

```
using System;
```

```
namespace ParameterizedConstructorExample {
```

```
class Geek {
```

```
    // data members of the class.
```

```
    String name;
```

```
    int id;
```

```
    // parameterized constructor would
```

```
    // initialize data members with
```

```
    // the values of passed arguments
```

```
    // while object of that class created.
```

```
    Geek(String name, int id)
```

```
    {
```

```
        this.name = name;
```

```
        this.id = id;
```

```
    }
```

```
    // Main Method
```

```
public static void Main()

{

    // This will invoke parameterized

    // constructor.

    Geek geek1 = new Geek("GFG", 1);

    Console.WriteLine("GeekName = " + geek1.name +

        " and GeekId = " + geek1.id);

}

}

}
```

Output :

GeekName = GFG and GeekId = 1

Copy Constructor

This constructor creates an object by copying variables from another object. Its main use is to initialize a new instance to the values of an existing instance.

Example :

C#

```
// C# Program to illustrate calling
```

```
// a Copy constructor

using System;

namespace copyConstructorExample {

class Geeks {

    private string month;

    private int year;

    // declaring Copy constructor

    public Geeks(Geeks s)

    {

        month = s.month;

        year = s.year;

    }

    // Instance constructor

    public Geeks(string month, int year)

    {

        this.month = month;

        this.year = year;

    }

    // Get details of Geeks
```

```

public string Details

{

    get

    {

        return "Month: " + month.ToString() +

            "\nYear: " + year.ToString();

    }

}

// Main Method

public static void Main()

{

    // Create a new Geeks object.

    Geeks g1 = new Geeks("June", 2018);

    // here is g1 details is copied to g2.

    Geeks g2 = new Geeks(g1);

    Console.WriteLine(g2.Details);

}

}

}

```

Output :

Month: June

Year: 2018

Private Constructor

If a constructor is created with private specifier is known as Private Constructor. It is not possible for other classes to derive from this class and also it's not possible to create an instance of this class.

Points To Remember :

- It is the implementation of a singleton class pattern.
- use private constructor when we have only static members.
- Using private constructor, prevents the creation of the instances of that class.

Example :

C#

```
// C# Program to illustrate calling

// a Private constructor

using System;

namespace privateConstructorExample {

    public class Geeks {

        // declare private Constructor

        private Geeks()

        {

        }

        // declare static variable field
```



```
public static int count_geeks;

// declare static method

public static int geeks_Count()

{

    return ++count_geeks;

}

// Main Method

public static void Main()

{

    // If you uncomment the following

    // statement, it will generate

    // an error because the constructor

    // is inaccessible:

    // Geeks s = new Geeks(); // Error

    Geeks.count_geeks = 99;

    // Accessing without any

    // instance of the class

    Geeks.geeks_Count();

    Console.WriteLine(Geeks.count_geeks);
```

```
// Accessing without any  
  
// instance of the class  
  
Geeks.geeks_Count();  
  
Console.WriteLine(Geeks.count_geeks);  
  
}  
  
}  
  
}
```

Output :

100

101

Static Constructor

Static Constructor has to be invoked only once in the class and it has been invoked during the creation of the first reference to a static member in the class. A static constructor is initialized static fields or data of the class and to be executed only once.

Points To Remember :

- It can't be called directly.
- When it is executing then the user has no control.
- It does not take access modifiers or any parameters.
- It is called automatically to initialize the class before the first instance created.

Example :

C#

```
// C# Program to illustrate calling
```

```
// a Static constructor

using System;

namespace staticConstructorExample {

    class geeks {

        // It is invoked before the first

        // instance constructor is run.

        static geeks()

        {

            // The following statement produces

            // the first line of output,

            // and the line occurs only once.

            Console.WriteLine("Static Constructor");

        }

        // Instance constructor.

        public geeks(int i)

        {

            Console.WriteLine("Instance Constructor " + i);

        }

    }

}
```

```
// Instance method.

public string geeks_detail(string name, int id)

{

    return "Name:" + name + " id:" + id;

}

// Main Method

public static void Main()

{

    // Here Both Static and instance

    // constructors are invoked for

    // first instance

    geeks obj = new geeks(1);

    Console.WriteLine(obj.geeks_detail("GFG", 1));

    // Here only instance constructor

    // will be invoked

    geeks obj1 = new geeks(2);

    Console.WriteLine(obj1.geeks_detail("GeeksforGeeks", 2));

}
```

```
}  
  
}
```

Output :

Static Constructor

Instance Constructor 1

Name:GFG id:1

Instance Constructor 2

Name:GeeksforGeeks id:2

C# Destructor

A destructor works opposite to constructor, It destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

Note: C# destructor cannot have parameters. Moreover, modifiers can't be applied on destructors.

C# Constructor and Destructor Example

Let's see an example of constructor and destructor in C# which is called automatically.

```
1. using System;  
2. public class Employee  
3. {  
4.     public Employee()  
5.     {  
6.         Console.WriteLine("Constructor Invoked");  
7.     }  
8.     ~Employee()  
9.     {  
10.        Console.WriteLine("Destructor Invoked");  
11.    }  
12. }  
13. class TestEmployee{  
14.     public static void Main(string[] args)
```

```
15.    {  
16.        Employee e1 = new Employee();  
17.        Employee e2 = new Employee();  
18.    }  
19. }
```

Output:

```
Constructor Invoked  
Constructor Invoked  
Destructor Invoked  
Destructor Invoked
```

Note: Destructor can't be public. We can't apply any modifier on destructors.

C# Operator Overloading

Overloading is generally defined as a process of implementing popular

Object-oriented programming concepts such as Polymorphism, which means one name having different forms and implementations. It allows the variables or the object to take different kinds of forms while executing the code. It is mainly used when you want the method property not to be similar to the given arguments, rather want a different order of execution when one name can be used with different types of execution methods and properties. This can be achieved in a program with different types of parameters and their count. There are various types of operator overloading methods in C#. You will learn all such methods in this tutorial conveniently.

Operator Overloading Techniques in C#

Operator overloading in C# can be done using different forms of operators. But before proceeding to the techniques, let's consider the validations of operators and how are they used while performing operator overloading.

Operators	Overloadability
+, -, *, /, %, &, , <,>, >>	All C# binary operators can be overloaded.
+, -, !, ~, ++, --, true, false	All C# unary operators can be overloaded.
==, !=, <, >, <=, >=	All relational operators can be overloaded, but only as pairs.
&&,	They can't be overloaded.
[] (Array index operator)	They can't be overloaded.
() (Conversion operator)	They can't be overloaded.
+=, -=, *=, /=, %=	These compound assignment operators can be overloaded. But in C#, these operators are automatically overloaded when the respective binary operator is overloaded.
=, ., ?:, ->, new, is, as, sizeof	These operators can't be overloaded in C#.

Syntax

1. **public static** classname operator op (parameters)
2. {
3. // Code
4. }
- 5.
6. For Unary Operator
- 7.
8. **public static** classname operator op (t)
9. {
10. // Code
11. }
- 12.
13. For Binary Operator
- 14.
15. **public static** classname operator op (t1, t2)
16. {
17. // Code
18. }

The operator is usually a **keyword** that is used while implementing operator overloading. It is important to note that the return type of the overloaded operator cannot be kept **void**. It means that while performing operator overloading, the preference is always in the over-defined implementations' hands. There is no preference given to predefined implementations.

Unary Operator Overloading

The general forms of operator overloading in the case of unary operator follows the syntax given below.

1. **public static** return_type operator op (Type t)
2. {
3. // Statements
4. }

Here, the return type can be anything, but it should not be void for the operators like **+**, **-**, **~** and **.(dot)**. The return type of this operator should be either an **integer** or a **Boolean** type. It is important to note that the Boolean operators roll out true and false and hence can be overloaded only as pairs. A compilation error would occur if not done because a class generally declares this operator without declaring the other.

1. using System;
2. **class** Complex
3. {
4. **private int** x;
5. **private int** y;
6. **public** Complex()
7. {
8. }
9. **public** Complex(**int** i, **int** j)
10. {
11. x = i;
12. y = j;
13. }
14. **public void** ShowXY()
15. {
16. Console.WriteLine("{0} {1}", x, y);
17. }
18. **public static** Complex operator -(Complex c)
19. {
20. Complex temp = **new** Complex();
21. temp.x = -c.x;
22. temp.y = -c.y;
23. **return** temp;
24. }
25. }
26. **class** MyClient
27. {
28. **public static void** Main()
29. {


```

30.     Complex c1 = new Complex(10, 20);
31.     c1.ShowXY(); // displays 10 & 20
32.     Complex c2 = new Complex();
33.     c2.ShowXY(); // displays 0 & 0
34.     c2 = -c1;
35.     c2.ShowXY(); // displays -10 & -20
36. }
37. }

```

Binary Operator Overloading

To overload a binary operator, you must seek two arguments. You need to ensure that one of the operators should be of type class or struct where the operator is defined. As previously discussed, a binary operator cannot return void, but it can return all other types of values while implementing overloading. The general form of operator overloading in binary operators can be shown using the following syntax.

```

1.  public static return_type operator op (Type1 t1, Type2 t2)
2.  {
3.  //Statements
4.  }

```

Consider the following example code that shows how binary operator overloading works.

```

1.  using System;
2.  class Complex
3.  {
4.      private int x;
5.      private int y;
6.      public Complex()
7.      {
8.      }
9.      public Complex(int i, int j)
10.     {
11.         x = i;
12.         y = j;
13.     }
14.     public void ShowXY()
15.     {
16.         Console.WriteLine("{0} {1}", x, y);
17.     }
18.     public static Complex operator +(Complex c1, Complex c2)
19.     {
20.         Complex temp = new Complex();
21.         temp.x = c1.x + c2.x;
22.         temp.y = c1.y + c2.y;
23.         return temp;
24.     }
25. }

```

```

24. }
25. class MyClient
26. {
27.     public static void Main()
28.     {
29.         Complex c1 = new Complex(10, 20);
30.         c1.ShowXY(); // displays 10 & 20
31.         Complex c2 = new Complex(20, 30);
32.         c2.ShowXY(); // displays 20 & 30
33.         Complex c3 = new Complex();
34.         c3 = c1 + c2;
35.         c3.ShowXY(); // displays 30 & 50
36.     }
37. }

```

It is important to keep a note that operators such as ==, !=, <>, <=, >= are only overloaded as pairs. When a binary arithmetic operator is overloaded using them, the assignment operators will automatically get overloaded. For instance, if you overload the + operator, it gets implicitly overloaded as += operator too.

Operator overloading & Inheritance

Another instance is that even if you declare the overloaded operators as static, they will inherit the derived class. This happens because the operator's declaration requires it to be struct or class where the operator is being declared. It facilitates the **operator's signature**. Thus, it is not possible for an operator previously declared in the derived class to hide an operator present in the parent class already. Hence, the new modifier is not an option to consider since they are not allowed while declaring the operator. This instance can be shown using the following code snippet given below.

```

1. using System;
2. class Complex
3. {
4.     private int x;
5.     private int y;
6.     public Complex()
7.     {
8.     }
9.     public Complex(int i, int j)
10.    {
11.        x = i;
12.        y = j;
13.    }
14.    public void ShowXY()
15.    {
16.        Console.WriteLine("{0} {1}", x, y);
17.    }

```

```

18. public static Complex operator +(Complex c1, Complex c2)
19. {
20.     Complex temp = new Complex();
21.     temp.x = c1.x + c2.x;
22.     temp.y = c1.y + c2.y;
23.     return temp;
24. }
25. }
26. class MyComplex: Complex
27. {
28.     private double x;
29.     private double y;
30.     public MyComplex(double i, double j)
31.     {
32.         x = i;
33.         y = j;
34.     }
35.     public MyComplex()
36.     {
37.     }
38.     public new void ShowXY()
39.     {
40.         Console.WriteLine("{0} {1}", x, y);
41.     }
42. }
43. class MyClient
44. {
45.     public static void Main()
46.     {
47.         MyComplex mc1 = new MyComplex(1.5, 2.5);
48.         mc1.ShowXY();
49.         MyComplex mc2 = new MyComplex(3.5, 4.5);
50.         mc2.ShowXY();
51.         MyComplex mc3 = new MyComplex();
52.         //mc3 = mc1 + mc2;
53.         //mc3.ShowXY();
54.     }
55. }

```

Equality Operator Overloading

You might have already been familiar with all the user-defined classes that inherit `System.Object` from the `System.Object.Equals()` method by default, in C#. The reference-based comparison is provided by the `Equals()` method. But there is a possibility that this method can override the methods residing inside the class defined by the user. Hence, value-based comparison can be easily achieved by using this method. This is how the Equality operator is

operated. To overload this operator, you need to follow the following code snippet shown below.

```
1. using System;
2. class Complex
3. {
4.     private int x;
5.     private int y;
6.     public Complex()
7.     {
8.     }
9.     public Complex(int i, int j)
10.    {
11.        x = i;
12.        y = j;
13.    }
14.    public void ShowXY()
15.    {
16.        Console.WriteLine("{0} {1}", x, y);
17.    }
18. }
19. class MyClient
20. {
21.     public static void Main()
22.     {
23.         Complex c1 = new Complex(10, 20);
24.         c1.ShowXY(); // displays 10 & 20
25.         Complex c2 = new Complex(10, 20);
26.         c2.ShowXY(); // displays 10 & 20
27.         Complex c3 = c2;
28.         c3.ShowXY(); // displays 10 & 20
29.         if (c1.Equals(c2))
30.             Console.WriteLine("OK");
31.         else
32.             Console.WriteLine("NOT OK");
33.         if (c2.Equals(c3))
34.             Console.WriteLine("OK1");
35.     }
36. }
```

Overloaded operators are functions with special names the keyword **operator** followed by the symbol for the operator being defined. similar

to any other function, an overloaded operator has a return type and a parameter list.

For example, go through the following function –

```
public static Box operator+ (Box b, Box c) {  
    Box box = new Box();  
    box.length = b.length + c.length;  
    box.breadth = b.breadth + c.breadth;  
    box.height = b.height + c.height;  
    return box;  
}
```

The above function implements the addition operator (+) for a user-defined class Box. It adds the attributes of two Box objects and returns the resultant Box object.

Implementing the Operator Overloading

The following program shows the complete implementation –

```
using System;  
  
namespace OperatorOvlApplication {  
    class Box {  
        private double length; // Length of a box  
        private double breadth; // Breadth of a box  
        private double height; // Height of a box  
  
        public double getVolume() {  
            return length * breadth * height;  
        }  
        public void setLength( double len ) {  
            length = len;  
        }  
        public void setBreadth( double bre ) {  
            breadth = bre;  
        }  
        public void setHeight( double hei ) {  
            height = hei;  
        }  
  
        // Overload + operator to add two Box objects.  
        public static Box operator+ (Box b, Box c) {  
            Box box = new Box();  
            box.length = b.length + c.length;  
            box.breadth = b.breadth + c.breadth;  
            box.height = b.height + c.height;  
            return box;  
        }  
    }  
}
```

```

        box.breadth = b.breadth + c.breadth;
        box.height = b.height + c.height;
        return box;
    }
}

class Tester {
    static void Main(string[] args) {
        Box Box1 = new Box(); // Declare Box1 of type Box
        Box Box2 = new Box(); // Declare Box2 of type Box
        Box Box3 = new Box(); // Declare Box3 of type Box
        double volume = 0.0; // Store the volume of a box here

        // box 1 specification
        Box1.setLength(6.0);
        Box1.setBreadth(7.0);
        Box1.setHeight(5.0);

        // box 2 specification
        Box2.setLength(12.0);
        Box2.setBreadth(13.0);
        Box2.setHeight(10.0);

        // volume of box 1
        volume = Box1.getVolume();
        Console.WriteLine("Volume of Box1 : {0}", volume);

        // volume of box 2
        volume = Box2.getVolume();
        Console.WriteLine("Volume of Box2 : {0}", volume);

        // Add two object as follows:
        Box3 = Box1 + Box2;

        // volume of box 3
        volume = Box3.getVolume();
        Console.WriteLine("Volume of Box3 : {0}", volume);
        Console.ReadKey();
    }
}

```

When the above code is compiled and executed, it produces the following result –

Volume of Box1 : 210

Volume of Box2 : 1560

Volume of Box3 : 5400